Listing 12.18 Configuring security policies and JWT authentication for Order Service

```
@EnableWebFluxSecurity                                          ❶
public class SecurityConfig {

  @Bean
  SecurityWebFilterChain filterChain(ServerHttpSecurity http) {
    return http
      .authorizeExchange(exchange -> exchange
        .anyExchange().authenticated()                          ❷
      )
      .oauth2ResourceServer(                                    ❸
        ServerHttpSecurity.OAuth2ResourceServerSpec::jwt)
      .requestCache(requestCacheSpec ->                         ❹
        requestCacheSpec.requestCache(NoOpServerRequestCache.getInstance()))
      .csrf(ServerHttpSecurity.CsrfSpec::disable)               ❺
      .build();
  }
}
```

❶ Enables Spring WebFlux support for Spring Security

❷ All requests require authentication.

❸ Enables OAuth2 Resource Server support using the default configuration based on JWT (JWT authentication)

❹ Each request must include an Access Token, so there's no need to keep a session cache alive between requests. We want it to be stateless.

❺ Since the authentication strategy is stateless and doesn't involve a browser-based client, we can safely disable the CSRF protection.

Let's check if this works. First, we need to run the backing services (Polar UI, Keycloak, Redis, RabbitMQ, and PostgreSQL). Open a Terminal window, navigate to the folder where you keep your Docker Compose configuration (polar-deployment/docker) and run the following:

```
$ docker-compose up -d polar-ui polar-keycloak polar-redis \
    polar-rabbitmq polar-postgres
```

Then run Edge Service, Catalog Service, and Order Service on the JVM (`./gradlew bootRun` from each project). Finally, open a browser